Chapter 7

Linear regression with a single predictor

As discussed in Chapter 1, regression is fundamentally a technology for predicting an outcome y from inputs $x_1, x_2, \ldots$. In this chapter we introduce regression in the simple (but not trivial) case of a linear model predicting a continuous y from a single continuous x, thus fitting the model $y_i = a + bx_i +$ error to data $(x_i, y_i), i = 1, \ldots, n$. We demonstrate with an applied example that includes the steps of fitting the model, displaying the data and fitted line, and interpreting the fit. We then show how to check the fitting procedure using fake-data simulation, and the chapter concludes with an explanation of how linear regression includes simple comparison as a special case.

7.1 Example: predicting presidential vote share from the economy

Example: Elections and the economy

Figure 7.1 tells a story of elections and the economy. It's based on the "bread and peace" model created by political scientist Douglas Hibbs to forecast elections based solely on economic growth (with corrections for wartime, notably Adlai Stevenson's exceptionally poor performance in 1952 and Hubert Humphrey's loss in 1968, years when Democratic presidents were presiding over unpopular wars). Better forecasts are possible using additional information (most notably, incumbency and opinion polls), but what is impressive here is that this simple model does pretty well.

Fitting a linear model to data

Figure 7.2 displays the economy and elections data as a prediction problem, with a fitted line predicting y (vote share) from x (economic performance). The data are in the file `hibbs.dat`, with the incumbent party's vote percentage of the two-party vote coded as `vote` and average personal income growth in the previous years coded as `growth`. We first read in and display the data:[1]

```
hibbs <- read.table("hibbs.dat", header=TRUE)
plot(hibbs$growth, hibbs$vote, xlab="Economic growth",
  ylab="Incumbent party's vote share")
```

Now we fit and display the model:

```
M1 <- stan_glm(vote ~ growth, data=hibbs)
print(M1)
```

In Chapter 8 we discuss the fitting procedure. For now, we work with the fitted model assuming it summarizes the data we have.

Understanding the fitted model

This is what we see in the R console after typing `print(M1)`:

[1] Data and code for this example are in the folder `ElectionsEconomy`.

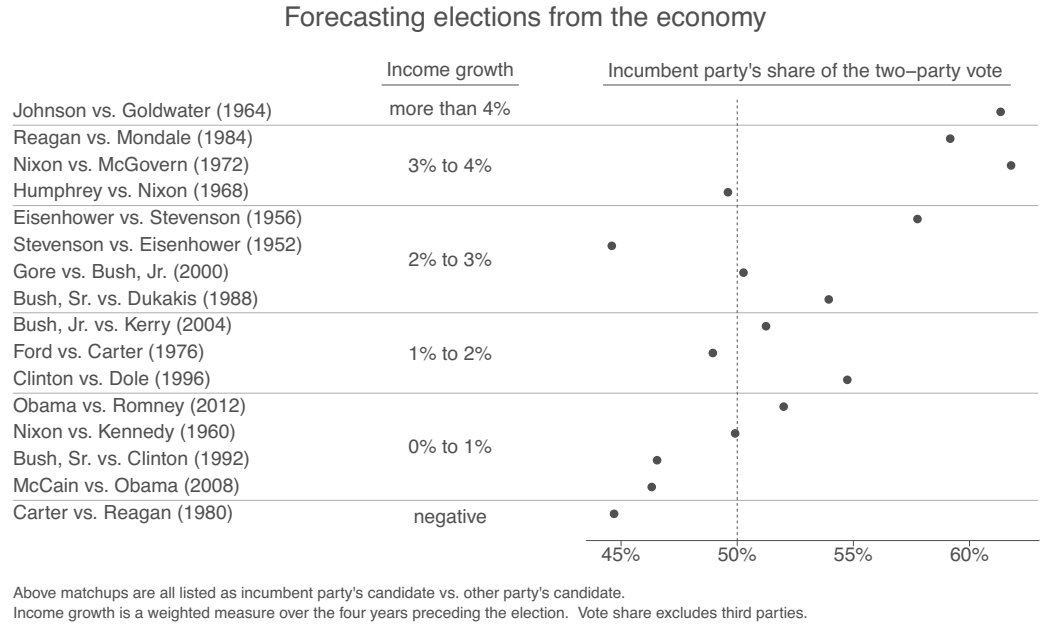


Figure 7.1 *Douglas Hibbs's "bread and peace" model of voting and the economy. Presidential elections since 1952 are listed in order of the economic performance at the end of the preceding administration (as measured by inflation-adjusted growth in average personal income). Matchups are listed as incumbent party's candidate versus other party's candidate. The better the economy was performing, the better the incumbent party's candidate did, with the biggest exceptions being 1952 (Korean War) and 1968 (Vietnam War).*

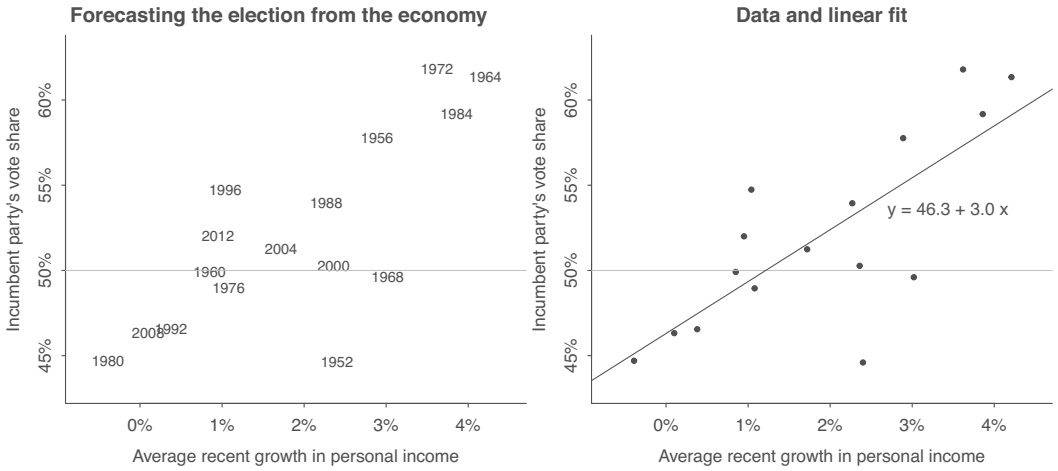


Figure 7.2 *(a) Predicting elections from the economy: the data in Figure 7.1 expressed as a scatterplot, with one data point for each year, (b) the data with the linear fit, $y = 46.3 + 3.0x$. Repeated from Figure 1.1.*

```
            Median MAD_SD
(Intercept) 46.3    1.7
growth       3.0    0.7

Auxiliary parameter(s):
      Median MAD_SD
sigma 3.9    0.7
```

In this example, `vote` is the outcome and `growth` is the predictor. By default the fit includes an intercept term: the line is $y = a + bx$, not simply $y = bx$.

To fit a line with zero intercept, $y = bx$, use the expression, `stan_glm(vote ~ -1 + growth)`; the `-1` tells R to exclude the intercept or constant term from the fit. In this example such a model would make no sense, but there are cases where it can be appropriate to fit a regression with no intercept, if you want the model to constrain the predicted value of y to be zero when $x = 0$.

To continue with the model we have fit, next is a table showing the estimates (`Median`) and uncertainties (`MAD_SD`) for the two coefficients, `Intercept` and `growth`, and the residual standard deviation `sigma`.

From this information, the *fitted line* is $y = 46.3 + 3.0x$:

- At $x = 0$ (zero economic growth), the incumbent party's candidate is predicted to receive 46.3% of the vote (that is, to lose). This makes sense: in recent history, zero growth represents poor performance, and a party that presides over this poor economy will be punished.
- Each percentage point of economic growth that is larger than 0 corresponds to an expected vote share for the incumbent party that is 3.0 percentage points higher than 46.3%. This jibes with the range of data and the line on the graph.
- The standard errors for the coefficients are small. We don't usually care so much about the standard error on the intercept, but the standard error on the slope is a measure of the uncertainty in this estimate. In this case, the standard error is 0.7, so the data are roughly consistent with a slope in the range 3.0 ± 0.7, and the 95% interval is $[3.0 \pm 2 * 0.7] = [1.6, 4.4]$. The interval is well separated from zero, indicating that, if the data had been generated from a model whose true slope is 0, it would be very unlikely to get an estimated slope coefficient this large.
- The estimated *residual standard deviation*, σ, is 3.9. That is, the linear model predicts the election outcome to within about 3.9 percentage points. Roughly 68% of the outcomes will fall between ± 3.9 of the fitted line. This makes sense. The predictions from the model are informative but not determinative. To put it another way: the model doesn't exactly predict each observation. Based on a linear model that only takes into account the economic growth, there is a limit to the accuracy with which you can predict an election.

One reason that Hibbs (and others) were confident in predicting a Democratic victory in 2008 could be seen from this linear model. The equation is $y = 46.3 + 3.0x$. In the years leading up to the 2008 election, the economic growth was approximately 0.1%, or $x = 0.1$. This linear model yields a prediction of $y = 46.6$, or an estimate of 46.6% of the popular vote going to the incumbent party, and thus a predicted 53.4% for Barack Obama, as the Republicans were the incumbent party in that year. The model fit when Obama was running for president was not identical to the model fit above, as it did not have the data points for 2008 and 2012. But it was similar, as can be seen by re-fitting the regression excluding those two elections.

Graphing the fitted regression line

We complete Figure 7.2b by adding the fitted line to the scatterplot we have already made:

```
abline(coef(M1))
```

The `coef` function returns the estimated coefficients of the linear model. The first coefficient is the intercept and the second is the slope for `growth`, which can be seen by typing `coef(M1)`. By plugging in these values into the `abline()` function, we are able to add the fitted model to the graph. The vertical discrepancies between the points and the line indicate the imperfections of the fitted model.

Using the model to predict

We fit the above model in advance of the 2016 presidential election of Democrat Hillary Clinton vs. Republican Donald Trump. At the time of fitting the model, the model's economic predictor—the

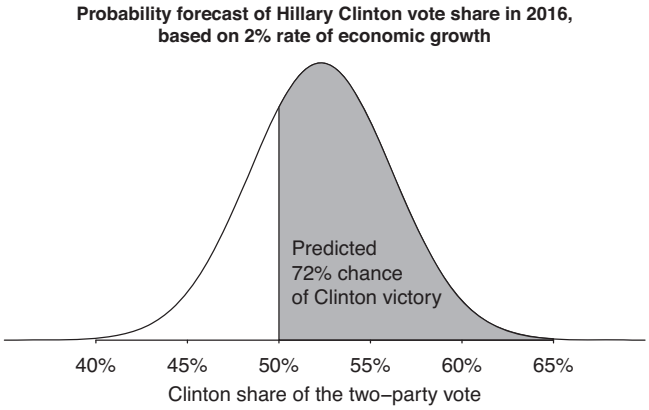


Figure 7.3 *Forecast distribution for Hillary Clinton's percentage of the two-party vote in 2016 based on an economic growth rate of 2%. The curve shows a normal distribution centered at the point forecast of* $46.3 + 3.0 * 2.0 = 52.3$ *and with standard deviation of 3.9, the estimated uncertainty based on the model fit. The shaded area indicates the probability (based on the model) that Clinton wins, which is* $1 - \text{pnorm}(50, 52.3, 3.9)$*, or 0.72.*

weighted average of economic growth of the previous four years—was approximately 2%. What, then, was Clinton's forecast vote percentage? The linear model predicted $46.3 + 3.0 * 2.0 = 52.3$. This forecast vote percentage does not alone tell us how likely it is that Clinton would win the popular vote in 2016. We also need some assessment of the uncertainty in that prediction.

Figure 7.3 shows the probability distribution of this forecast, which is centered on 52.3 with a standard deviation of 3.9. In constructing the regression model, we were assuming a normal approximation for the errors, which is conventional and also makes sense for the reasons discussed in Section 3.5 regarding the Central Limit Theorem, implicitly modeling errors as sums of many small independent components. We are very slightly understating uncertainty here by considering the fitted parameters of the regression model as known; in Chapter 9, we fix this by using simulation to combine the uncertainties in estimation and prediction.

To continue with the simple illustrative calculation, we can use the forecast of *vote share* to estimate the probability that Clinton would win the popular vote. She wins if her vote share exceeds 50%; this is the shaded area in the curve in Figure 7.3. This is a probability distribution, so the total area under the curve is 1. To find the shaded area in R, type:

```
1 - pnorm(50, 52.3, 3.9)
```

This returns 0.72. According to this model, there was a 72% chance that Clinton would win the popular vote, running in 2016 with economic growth of 2%.

If the goal is to predict the winner, why predict vote share at all? As shown in Figure 7.2, Hibbs's model predicts the election outcome as a continuous vote share, some number between 0 and 100% (in practice, between 40% and 60%). Why not instead use the economic data to directly predict who wins the election?

The reason for not simply predicting the winner can be seen by considering three sorts of elections:

- *Evenly divided elections* such as 1960 (Kennedy vs. Nixon) that were essentially tied. It would be unreasonable and meaningless to attempt to predict the winner of such an election. The best you can hope to do is to predict the vote percentages—that is, in these cases, to predict that the election is likely to be close.
- *Somewhat competitive elections* such as 2008 (Obama vs. McCain) in which one side was predicted to win but the other side was perceived as having some chance of an upset. Here you would like to predict the vote margin and the probability of either candidate winning, rather than trying to make a deterministic call.

- *Landslides* such as 1984 (Reagan vs. Mondale). Here there is nothing impressive about predicting the winner, but it is still a challenge to accurately predict the percentage of the vote received by the two candidates.

In any of these cases, a prediction of the vote share is more informative than simply predicting the winner.

In the event, Clinton received 51.1% of the popular vote, well within the forecast distribution, but she lost the electoral college, which is decided by which states are won by which candidates. To forecast the electoral-vote winner, one must predict the vote in each state, which can be done using a multilevel model, but this is beyond the scope of this book.

7.2 Checking the model-fitting procedure using fake-data simulation

The preceding example is simple enough that we can just make a graph and see that the line goes through it, but more generally it is good practice to check our fits by performing them under controlled conditions where we know the truth. We demonstrate here with the election model.[2]

Step 1: Creating the pretend world

We start by assuming true values for all the parameters in the model. In this case, we have already fit a model to data so we shall proceed by pretending that these particular parameter values are the truth. That is, we assume the relation $y = 46.3 + 3.0x +$ error is true, with the errors drawn from a normal distribution with mean 0 and standard deviation 3.9. Then (using the predictors x that we already have from our dataset) we examine whether these predictors create a distribution of y that is consistent with our observed y.

```
a <- 46.3
b <- 3.0
sigma <- 3.9
x <- hibbs$growth
n <- length(x)
```

Step 2: Simulating fake data

We then simulate a vector y of fake data and put this all in a data frame:

```
y <- a + b*x + rnorm(n, 0, sigma)
fake <- data.frame(x, y)
```

Step 3: Fitting the model and comparing fitted to assumed values

The next step is to fit a regression to these data. The fitting makes no use of the assumed true values of α, β, and σ.

```
fit <- stan_glm(y ~ x, data=fake)
print(fit)
```

Here is the regression output:

```
            Median MAD_SD
(Intercept) 44.4    1.7
x            3.2    0.7
```

[2]Data and code for this example are in the folder `ElectionsEconomy`.

```
Auxiliary parameter(s):
      Median MAD_SD
sigma 4.0    0.8
```

Comparing the estimated coefficients to the assumed true values 46.3 and 3.0, the fit seems reasonable enough: the estimates are not exact but are within the margins of error.

We can perform this comparison more formally by extracting from the regression object the estimates and standard errors of the coefficients. Here for simplicity in programming we do this for just one of the coefficients, the slope b:

```
b_hat <- coef(fit)["x"]
b_se <- se(fit)["x"]
```

We then check whether the true value of b falls within the estimated 68% and 95% confidence intervals obtained by taking the estimate ± 1 or ± 2 standard errors (recall Figure 4.1):

```
cover_68 <- abs(b - b_hat) < b_se
cover_95 <- abs(b - b_hat) < 2*b_se
cat(paste("68% coverage: ", cover_68, "\n"))
cat(paste("95% coverage: ", cover_95, "\n"))
```

Step 4: Embedding the simulation in a loop

The confidence intervals worked once, but do they have the correct *coverage probabilities*—that is, do the intervals contain the true value the advertised percentage of the time, as in Figure 4.2? To check, we use the normal distribution; recall Figure 4.1. We can simulate the sampling distribution and thus compute the coverage of the confidence interval by embedding the data simulation, model fitting, and coverage checking in a loop and running 1000 times. This and other loops in this chapter could also be performed implicitly using the `replicate` function in R, as illustrated on pages 72 and 74. Here we code the loop directly:

```
n_fake <- 1000
cover_68 <- rep(NA, n_fake)
cover_95 <- rep(NA, n_fake)
for (s in 1:n_fake){
  y <- a + b*x + rnorm(n, 0, sigma)
  fake <- data.frame(x, y)
  fit <- stan_glm(y ~ x, data=fake, refresh=0)     # suppress output on console
  b_hat <- coef(fit)["x"]
  b_se <- se(fit)["x"]
  cover_68[s] <- abs(b - b_hat) < b_se
  cover_95[s] <- abs(b - b_hat) < 2*b_se
}
cat(paste("68% coverage: ", mean(cover_68), "\n"))
cat(paste("95% coverage: ", mean(cover_95), "\n"))
```

This takes a few minutes to run, and then the following appears on the console:

```
68% coverage:  0.628
95% coverage:  0.928
```

That is, `mean(cover_68)` = 63% and `mean(cover_95)` = 93%, not far from the nominal values of 0.68 and 0.95. The coverage is a bit low, in part because ± 1 and ± 2 are standard error bounds for the normal distribution, but with a sample size of only 16, our inferences should use the t distribution with 14 degrees of freedom. This is not a big deal, but just to be careful we can redo by checking the coverage of the appropriate t intervals:

```
n_fake <- 1000
cover_68 <- rep(NA, n_fake)
cover_95 <- rep(NA, n_fake)
t_68 <- qt(0.84, n - 2)
t_95 <- qt(0.975, n - 2)
for (s in 1:n_fake){
  y <- a + b*x + rnorm(n, 0, sigma)
  fake <- data.frame(x, y)
  fit <- stan_glm(y ~ x, data=fake, refresh=0)
  b_hat <- coef(fit)["x"]
  b_se <- se(fit)["x"]
  cover_68[s] <- abs(b - b_hat) < t_68 * b_se
  cover_95[s] <- abs(b - b_hat) < t_95 * b_se
}
cat(paste("68% coverage: ", mean(cover_68), "\n"))
cat(paste("95% coverage: ", mean(cover_95), "\n"))
```

This simulation gives the desired result that approximately 68% of the 68% intervals, and approximately 95% of the 95% intervals, contain the true parameter values.

7.3 Formulating comparisons as regression models

To connect to the basic statistical methods of Chapter 4, we show how simple averages and comparisons can be interpreted as special cases of linear regression. This more general formulation is helpful on a conceptual level to unify these seemingly different ideas, and it will help in practice as a key component in building larger models, as we show in the following chapters.

To express comparisons as regressions we need the concept of an *indicator variable*, which is a predictor that equals 1 or 0 to indicate whether a data point falls into a specified category. Indicator variables can be used for inputs with two categories (for example, an indicator for "male" that takes on the value 1 for men and 0 for women) or multiple categories (for example, indicators for "White," "Black," and "Hispanic," with each taking on the value of 1 for people in the specified category and zero otherwise).

For simplicity we here use fake data to demonstrate the connection between comparisons and regression on indicators.[3]

Estimating the mean is the same as regressing on a constant term

Example: Comparison and simple regression

Let's simulate 20 observations from a population with mean 2.0 and standard deviation 5.0:

```
n_0 <- 20
y_0 <- rnorm(n_0, 2.0, 5.0)
fake_0 <- data.frame(y_0)
print(y_0)
```

These are the numbers that arose in one particular simulation:

```
 -0.3   4.1  -4.9   3.3   6.4   7.2  10.7  -4.6   4.7   6.0   1.1  -6.7  10.2
  9.7   5.6   1.7   1.3   6.2  -2.1   6.5
```

Considering these as a random sample, we can estimate the mean of the population as `mean(y_0)` with standard error `sd(y_0)/sqrt(n_0)`. The result: an estimate of 3.3 with standard error 1.1.

We get the identical result using least squares regression on a constant term:

```
fit_0 <- stan_glm(y_0 ~ 1, data=fake_0,
  prior_intercept=NULL, prior=NULL, prior_aux=NULL)
print(fit_0)
```

[3]Code for this example is in the folder `Simplest`.

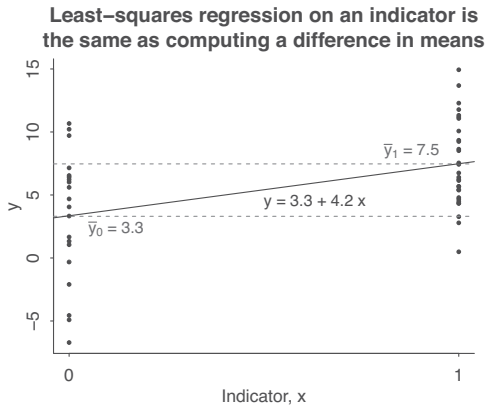


Figure 7.4 *Simulated-data example showing how regression on an indicator variable is the same as computing the difference in means between two groups.*

which yields:

```
             Median MAD_SD
(Intercept) 3.3    1.1

Auxiliary parameter(s):
      Median MAD_SD
sigma 5.3    0.9
```

As discussed in Chapters 8 and 9, the above settings assign a flat prior so that `stan_glm` will produce the classical least squares regression estimate. We discuss prior distributions more in Section 9.5; here we use the simple least-squares estimate to demonstrate the equivalence between the simple average and the intercept-only regression; in practice the default prior for `stan_glm` would give a very similar result.

From our simulations, we know that the true intercept for this regression is 2.0 and the residual standard deviation σ is 5.0. But this small sample size yields noisy estimates. The estimates of 3.3 and 5.3 are consistent with the true parameter values, given the standard errors of the fit.

Estimating a difference is the same as regressing on an indicator variable

Next add in a new group: 30 observations from a population with mean 8.0 and standard deviation 5.0:

```
n_1 <- 30
y_1 <- rnorm(n_1, 8.0, 5.0)
```

We can directly compare the averages in each group and compute the corresponding standard error:

```
diff <- mean(y_1) - mean(y_0)
se_0 <- sd(y_0)/sqrt(n_0)
se_1 <- sd(y_1)/sqrt(n_1)
se <- sqrt(se_0^2 + se_1^2)
```

In our particular simulation, these return the value 4.2 for the difference and 1.3 for its standard error. This result is consistent with the construction of our simulations in which the true population difference was 6.0.

Alternatively we can frame the problem as a regression by combining the data into a single vector, y, and then creating an *indicator variable* x, defined as,

$$x_i = \begin{cases} 0 & \text{if observation } i \text{ is in group 0} \\ 1 & \text{if observation } i \text{ is in group 1.} \end{cases}$$

In R:

```
n <- n_0 + n_1
y <- c(y_0, y_1)
x <- c(rep(0, n_0), rep(1, n_1))
fake <- data.frame(x, y)
fit <- stan_glm(y ~ x, data=fake, prior_intercept=NULL, prior=NULL, prior_aux=NULL)
print(fit)
```

The result:

```
            Median MAD_SD
(Intercept) 3.3    1.1
x           4.2    1.2

Auxiliary parameter(s):
      Median MAD_SD
sigma 4.7    0.5
```

The estimate of the slope, 4.2, is identical to the difference in means, $\bar{y}_1 - \bar{y}_0$, as it must be for this simple model. We discuss priors more fully in Chapter 9. The standard error is nearly identical but differs slightly because the regression model estimates a single residual standard deviation parameter, as compared to the difference calculation which uses separate values of se_0 and se_1.

Figure 7.4 visually displays the equivalence of the two estimates: the regression on an indicator variable (shown by the fitted line) and the comparison of the two means (shown by the difference between the two dotted lines). In this simple setting with no other predictors, the least squares line will go through the points $(0, \bar{y}_0)$ and $(1, \bar{y}_1)$, so the slope of the line is simply the difference, $\bar{y}_1 - \bar{y}_0$. In more complicated settings, we can use indicator variables to compare groups, adjusting for other predictors.

The point of doing all this using fake-data simulation is, first, to directly check that the direct comparison and the regression give the same answer and, second, to understand the properties of statistical fits using a general tool that will continue to be helpful in more complicated settings. In Section 10.4 we discuss the use of indicator variables in regression models with multiple predictors.

7.4 Bibliographic note

The material in this and the following chapters on linear regression is covered in many books which specialize in various aspects of the problem. Harrell (2001) covers applied and conceptual issues in regression. Fox (2002) is a good introductory text on fitting and displaying regressions in R.

For research on economic performance and incumbent party vote for president, see Hibbs, Rivers, and Vasilatos (1982) with updates by Hibbs (2000, 2012), and also Rosenstone (1983), Fair (1978), and Wlezien and Erikson (2004, 2005).

7.5 Exercises

7.1 *Regression predictors*: In the election forecasting example of Section 7.1, we used inflation-adjusted growth in average personal income as a predictor. From the standpoint of economics, it makes sense to adjust for inflation here. But suppose the model had used growth in average personal income, not adjusting for inflation. How would this have changed the resulting regression? How would this change have affected the fit and interpretation of the results?

7.2 *Fake-data simulation and regression*: Simulate 100 data points from the linear model, $y = a + bx +$ error, with $a = 5$, $b = 7$, the values of x being sampled at random from a uniform distribution on the range $[0, 50]$, and errors that are normally distributed with mean 0 and standard deviation 3.

(a) Fit a regression line to these data and display the output.

(b) Graph a scatterplot of the data and the regression line.

(c) Use the `text` function in R to add the formula of the fitted line to the graph.

7.3 *Fake-data simulation and fitting the wrong model*: Simulate 100 data points from the model, $y = a + bx + cx^2 +$ error, with the values of x being sampled at random from a uniform distribution on the range [0, 50], errors that are normally distributed with mean 0 and standard deviation 3, and a, b, c chosen so that a scatterplot of the data shows a clear nonlinear curve.

(a) Fit a regression line `stan_glm(y ~ x)` to these data and display the output.

(b) Graph a scatterplot of the data and the regression line. This is the best-fit linear regression. What does "best-fit" mean in this context?

7.4 *Prediction*: Following the template of Section 7.1, find data in which one variable can be used to predict the other, then fit a linear model and plot it along with the data, then display the fitted model and explain in words as on page 95. Use the model to obtain a probabilistic prediction for new data, and evaluate that prediction, as in the last part of Section 7.1.

7.5 *Convergence as sample size increases*: Set up a simulation study such as in Section 7.2, writing the entire simulation as a function, with one of the arguments being the number of data points, n. Compute the simulation for $n = 10$, 30, 100, 300, 1000, 3000, 10 000, and 30 000, for each displaying the estimate and standard error. Graph these to show the increasing stability as n increases.

7.6 *Formulating comparisons as regression models*: Take the election forecasting model and simplify it by creating a binary predictor defined as $x = 0$ if income growth is less than 2% and $x = 1$ if income growth is more than 2%.

(a) Compute the difference in incumbent party's vote share on average, comparing those two groups of elections, and determine the standard error for this difference.

(b) Regress incumbent party's vote share on the binary predictor of income growth and check that the resulting estimate and standard error are the same as above.

7.7 *Comparing simulated data to assumed parameter values*:

(a) Simulate 100 data points from the model, $y = 2 + 3x +$ error, with predictors x drawn from a uniform distribution from 0 to 20, and with independent errors drawn from the normal distribution with mean 0 and standard deviation 5. Save x and y into a data frame called `fake`. Fit the model, `stan_glm(y ~ x, data=fake)`. Plot the data and fitted regression line.

(b) Check that the estimated coefficients from the fitted model are reasonably close to the assumed true values. What does "reasonably close" mean in this context?

7.8 *Sampling distribution*: Repeat the steps of the previous exercise 1000 times (omitting the plotting). Check that the coefficient estimates are approximately unbiased, that their standard deviations in the sampling distribution are approximately equal to their standard errors, and that approximately 95% of the estimate ± 2 standard error intervals contain the true parameter values.

7.9 *Interpretation of regressions*: Redo the election forecasting example of Section 7.1, but switching x and y, that is, predicting economic growth given the subsequent election outcome. Discuss the problems with giving a causal interpretation to the coefficients in this regression, and consider what this implies about any causal interpretations of the original regression fit in the chapter.

7.10 *Working through your own example*: Continuing the example from the final exercises of the earlier chapters, fit a linear regression with a single predictor, graph the data along with the fitted line, and interpret the estimated parameters and their uncertainties.